HEADLINE METRIC

4-state

Closed verdict taxonomy : PASS · WARN · FAIL · INCONCLUSIVE.

Meta-Swarm — Coordination Layer Report

Phase 2 close → Phase 3 start. Investor due-diligence brief.

ISSUED
2026-05-16

AUTHOR
Engineering Leadership · EFE

AUDIENCE
Technical investors — infrastructure /

CONFIDENTIAL · INVESTOR DUE DILIGENCE · Not for redistribution.

Built 2026-05-16

A LETTER TO THE READER

The coordination layer, in plain language.

This document is the companion to the platform's general engineering brief, and it is focused specifically on the part of the platform that coordinates the work of independent participants. We chose to write a separate brief because the coordination layer has a small number of distinctive properties that deserve their own treatment, and because some readers will be interested in the coordination layer specifically without wanting to read the whole engineering brief.

The coordination layer is the part of our platform that tells independent processes what to do, listens to what they answer, and combines their answers into a single outcome. The independent processes can be running anywhere; they can be written in any programming language; they can fail in any number of expected and unexpected ways. The coordination layer's job is to make their collective behaviour visible, reproducible, and independently verifiable.

This brief explains in detail how the coordination layer achieves those three goals. It is written in plain language for a technically literate reader who has not necessarily worked on distributed systems before, although the reader who has worked on distributed systems will recognise the architectural choices we describe as belonging to a small family of well-understood designs. Where we depart from the family in a meaningful way, we say so explicitly and explain why.

HOW TO READ THIS DOCUMENT

The first three parts describe what the coordination layer is and what it commits to. The middle parts describe how it works in detail. The closing parts describe how outsiders can verify its behaviour and where it fits in the broader landscape. A reader interested in only the investor-facing properties can read the first three parts and the closing in about twenty-five minutes.

What the coordination layer is

The shape of the protocol, the participants, and the outcomes we commit to.

02

PART TWO · THE SHAPE OF THE PROTOCOL

The shape of the conversation we mediate.

The coordination layer mediates a structured conversation between a central coordinator and a small number of independent participants. The conversation has a small number of well-defined phases, each phase has a single purpose, and the structure of the conversation is the same across every test exercise the platform runs. We describe the phases in this section, in the order they occur, so that a reader unfamiliar with the protocol can build an intuition for how it works.

The first phase is the registration phase. When a participant comes online, it announces itself to the coordinator and provides a small amount of structured information about itself: its identifier, its operating environment, and the version of the protocol it speaks. The coordinator records the registration and acknowledges it, or rejects the participant if the protocol version is incompatible with the coordinator's. The registration phase is short, typically taking less than a second on a modern network, and it establishes the basic trust relationship between the coordinator and the participant for the duration of the subsequent exercise.

The second phase, and the longest one, is the exercise phase. The coordinator sends a stream of small instructions to the participant, each instruction describing a single step of the test exercise. The participant performs each step, reports back its result, and waits for the next instruction. The instructions flow from coordinator to participant, the results flow from participant to coordinator, and both directions are part of the same connection so that the coordinator's view of which step is currently in flight is always consistent with the participant's. The exercise phase lasts as long as the specific test exercise requires, which can range from a few seconds to several hours depending on the exercise.

The third phase is the measurement phase. Throughout the exercise phase, the participant emits a stream of measurements describing what it observed about its own behaviour: response times, error rates, resource consumption, and a small number of exercise-specific properties. The measurements flow from participant to coordinator on a separate channel from the exercise instructions, so that the measurement traffic does not interfere with the exercise traffic. The coordinator collects the measurements and stores them for later aggregation.

The fourth phase is the voting phase, which only applies to exercises that require participants to agree on an outcome. The coordinator asks each participant to vote on what it believes the outcome should be, the participants vote, and the coordinator collects the votes. The vote is the place where a deliberately misbehaving participant would betray its misbehaviour, because the vote of a misbehaving participant will disagree with the votes of the well-behaved participants. The coordinator records the disagreement in a structured form that the outcome aggregator can later interpret.

The fifth and final phase is the withdrawal phase. When the exercise concludes, each participant withdraws cleanly, reporting any final state and releasing any resources it held during the exercise. The withdrawal phase is short, like the registration phase, and it returns the participant to its initial state ready for the next exercise.

PART TWO · WHY THE PROTOCOL HAS THIS SHAPE

Why these five phases, and not others.

A reader familiar with distributed-systems protocols may ask why we chose to organise the protocol into exactly these five phases, rather than into fewer phases or more. The answer involves a tradeoff that took us several rounds of design iteration to resolve, and we describe the tradeoff in this section because it explains a number of downstream decisions.

The competing pressures were two. On the one hand, we wanted the protocol to be small: a small protocol is easier to specify, easier to implement, easier to test exhaustively, and easier to keep stable across long time horizons. On the other hand, we wanted the protocol to be structured: a structured protocol gives each kind of participant-coordinator interaction its own well-defined boundaries, which makes it possible to reason about failures and to test each interaction independently.

Five phases turned out to be the smallest number we could justify while still satisfying the second requirement. We considered collapsing the registration and exercise phases into a single phase, but we found that the registration's failure modes are different enough from the exercise's failure modes that they benefit from independent treatment. We considered collapsing the exercise and measurement phases, but we found that the measurement stream needs to be on a separate channel from the exercise stream so that measurement back-pressure cannot stall the exercise or vice versa. We considered eliminating the voting phase entirely and inferring the outcome from the measurements alone, but we found that explicit voting is required for exercises in which participants might disagree, which is the kind of exercise our threat model most cares about.

Having settled on five phases, we have been careful not to add a sixth. Every proposal to add a phase has been evaluated against the alternative of expressing the new behaviour within an existing phase, and so far we have always found that the existing phases are sufficient. The protocol's stability commitment, which we described in the main brief, freezes the five-phase structure for at least the next six minor releases of the platform. Any future addition of a sixth phase would be a major release, with an explicit migration path for every existing participant.

PART TWO · THE CLOSED OUTCOME VOCABULARY

The four outcomes the platform commits to.

Every test exercise the platform runs produces an outcome that takes one of four values, and the platform has committed to never extending the vocabulary beyond those four. The four values are the platform's most visible external commitment, because every downstream consumer of the platform's outputs depends on the values being exactly these four. We describe each in this section, in enough detail that a consumer can build correct intuition about when each value will be emitted.

The first value, which we call simply *passing*, is emitted when every measurement produced by the exercise was within the tolerance budget specified for that exercise, and when no failure-injection hook caused the platform to operate outside its service-level commitments. The passing value is the default case for healthy exercises; it is the value emitted by the vast majority of the platform's runs.

The second value, which we call *warning*, is emitted when at least one measurement drifted from its reference value, but the drift remained within the tolerance budget specified for that measurement. The warning value tells downstream consumers that some property of the platform is moving, even though the property has not yet moved far enough to violate a commitment. The warning value is the platform's early-warning signal, and the absence of a warning over a sustained period is itself a positive signal about the platform's stability.

The third value, which we call *failing*, is emitted when at least one measurement breached its tolerance budget. The failing value tells downstream consumers that a commitment has been violated, and it is always accompanied by an explicit record of which measurement breached and by how much. The failing value is the platform's most consequential output, and it triggers the platform's audit discipline in addition to the normal outcome reporting.

The fourth value, which we call *inconclusive*, is emitted when the platform could not produce a definitive outcome because the exercise itself did not produce enough data to be conclusive. The inconclusive value is emitted, for example, when the platform's coordinator crashed mid-exercise, when the network between the coordinator and the participants partitioned irrecoverably, or when the underlying machinery the exercise depended on was unavailable. The inconclusive value tells downstream consumers that no judgement can be made about this run, and that the run should be retried before any action is taken.

The four values together form what we call a closed vocabulary, by which we mean that no other values will ever be emitted. A consumer can rely on this property and write code that handles exactly these four cases without worrying about future additions. The closed vocabulary is part of the platform's durability commitment, and extending it would require a major release with an explicit migration path. We have committed not to do this within the horizon of at least the next six minor releases.

The participants

Who participates in the platform's exercises, what they commit to, and how they can fail.

03

PART THREE · WHAT A PARTICIPANT IS

Participants, in detail.

Throughout this brief we have used the word *participant* to mean an independent process that talks to the platform's coordinator over the protocol we have just described. We now want to describe what kind of process a participant is in more detail, because the participants' properties shape what the platform can and cannot promise to downstream consumers.

A participant is any process that implements the coordination protocol on its server side. The protocol is defined by a structured-message format that describes the messages flowing in each direction and the rules under which messages may be exchanged. Any process that can speak the structured-message format and that observes the rules is a valid participant. We do not require participants to be written in any particular programming language, to run on any particular operating system, or to be operated by any particular organisation. The only requirement is that they speak the protocol correctly.

This openness is important because it means the coordinator does not need to trust the participants' implementations, only the protocol they share. If a participant misbehaves, the coordinator can detect the misbehaviour through the protocol's structure: a participant that violates the protocol's rules is identifiable as misbehaving regardless of why it is doing so. This is a different kind of trust relationship from the one in which the coordinator and the participants are all written by the same team; the federation design we described in the main brief is what allows us to maintain the looser relationship.

Participants are stateful, in the sense that each participant carries some state between phases of an exercise. The registration phase establishes some of that state; the exercise phase mutates it as instructions are processed; the measurement phase emits derived state to the coordinator. The withdrawal phase clears the state, returning the participant to its initial condition. We have designed the protocol so that no state is carried across exercises: each exercise begins with a fresh registration and ends with a clean withdrawal. This means that misbehaviour in one exercise cannot contaminate a subsequent exercise, which is an important property for the platform's audit posture.

PART THREE · THE FAILURE MODES WE TOLERATE

How participants can fail, and how we handle each.

Participants can fail in a number of distinct ways, and the platform is designed to tolerate each kind of failure in a way appropriate to the failure's shape. We describe the most important failure modes in this section, because the platform's tolerance of failures is one of its most carefully-tested properties and the kind of thing an investor doing diligence should understand in detail.

The simplest failure mode is participant loss: a participant that was alive at the start of an exercise stops responding before the exercise concludes. This can happen because the participant crashed, because the network between the participant and the coordinator partitioned, or because the participant deliberately withdrew without going through the proper withdrawal phase. The platform tolerates participant loss by detecting the loss within a bounded timeout, recording the loss in the exercise's structured record, and continuing the exercise with the remaining participants. The exercise's outcome reflects whether enough participants remained to make a conclusive judgement; if not, the outcome is inconclusive.

A more subtle failure mode is network partition: the network between two participants partitions, but each participant remains reachable from the coordinator. From the coordinator's point of view all participants are alive, but the participants cannot reach each other, which matters for exercises that require the participants to coordinate among themselves. The platform's tolerance for partition is exercised by a specific test exercise in the catalogue, and the exercise verifies that the platform's outcome aggregator correctly identifies which side of the partition is responsible for the failure.

A still more subtle failure mode is deliberate adversarial behaviour: a participant that is alive, reachable, and apparently well-behaved at the protocol level returns answers that are deliberately wrong. This failure mode is the one most often discussed in the academic literature on distributed systems, because it is the failure mode most resistant to ordinary detection. The platform tolerates adversarial behaviour through the voting phase we described earlier: when participants vote on an outcome, an adversarial participant's vote will disagree with the well-behaved participants' votes, and the coordinator records the disagreement in a structured form that the outcome aggregator can later interpret.

A fourth failure mode is clock skew: the participants' clocks disagree with each other and with the coordinator's clock, even though all parties are otherwise behaving correctly. Clock skew is exercised by a specific test exercise in which the coordinator deliberately introduces a skew of up to fifty milliseconds between the participants, and the exercise verifies that the platform's measurements correctly account for the skew when comparing timestamps across participants.

A fifth failure mode is network message reordering and duplication: the messages flowing between the coordinator and the participants arrive in a different order than they were sent, or arrive multiple times. This failure mode is exercised by another specific test exercise, and the exercise verifies that the platform's protocol correctly handles reordering and duplication without producing incorrect outcomes.

Each of these five failure modes has its own test exercise in the platform's catalogue, and each exercise has been calibrated against the specific failure pattern it injects. The exercises are run as part of the platform's continuous verification pipeline, which means that any regression in the platform's tolerance of any of these failure modes would be caught before the regression could ship.

The outcome aggregator

How we combine many per-exercise outcomes into a single overall outcome, and what mathematics we use.

04

PART FOUR · WHY AGGREGATION MATTERS

The aggregation problem, before the solution.

The platform's test catalogue contains ten exercises, and a single platform release runs all ten of them. Each exercise produces its own outcome, in the four-valued closed vocabulary we described earlier. The platform also needs to produce a single overall outcome that summarises the release as a whole. The operation that combines the ten per-exercise outcomes into one overall outcome is what we call the outcome aggregator, and we describe it in this part of the brief because the choice of aggregation rule has consequences that may not be obvious.

The most obvious aggregation rule, and the one we use, is to take the worst per-exercise outcome as the overall outcome. If any exercise was inconclusive, the overall outcome is inconclusive. Otherwise, if any exercise was failing, the overall outcome is failing. Otherwise, if any exercise was warning, the overall outcome is warning. Otherwise, the overall outcome is passing. This rule has the property that adding more exercises to the catalogue can only worsen the overall outcome, never improve it; a release that was passing under ten exercises will continue to be passing under eleven, but a release that was passing under ten might be warning or failing under eleven.

The choice of the worst-case aggregator is deliberate. The platform's external value depends on the credibility of its outcomes, and the worst way to compromise the credibility would be to produce an aggregated outcome that masks a failing individual exercise behind a sea of passing ones. The worst-case rule prevents this masking: any failing exercise contaminates the overall outcome, which forces the team to investigate before the release can claim a passing judgement.

We considered an alternative aggregator that uses the median rather than the worst case as the summary statistic, and we discussed this alternative in the main brief as one of our deferred work items. The median aggregator is more robust to outliers but less defensible in an audit context, and we have decided to keep the worst-case aggregator for the foreseeable future.

PART FOUR · WHY INCONCLUSIVE RANKS ABOVE FAILING

The subtle priority of inconclusive.

An attentive reader will notice that our aggregation rule ranks inconclusive above failing, in the sense that an exercise producing an inconclusive outcome contaminates the overall outcome more strongly than an exercise producing a failing outcome. This is the opposite of the intuitive ordering, in which failing might seem more severe than inconclusive. We chose this ordering deliberately, and we want to explain why because the explanation reveals something important about how we think about evidence.

A failing outcome is a known outcome. The platform ran the exercise to completion, observed that some tolerance budget was breached, and emitted a structured record of which budget was breached and by how much. The team can read the record, understand the failure, and take action. A failing outcome contains information.

An inconclusive outcome, by contrast, is an unknown outcome. The platform could not complete the exercise, for reasons that may or may not be understood, and emitted a structured record only of the fact that the exercise could not be concluded. The team can read the record, but cannot infer from it whether the underlying behaviour was correct or incorrect. An inconclusive outcome contains no information about the underlying behaviour, only about the platform's inability to measure it.

From the point of view of a downstream consumer trying to decide whether to trust the platform's outputs, an unknown outcome is therefore at least as bad as a failing outcome, and arguably worse. A failing outcome at least lets the consumer know what is wrong; an unknown outcome leaves them to guess. The aggregator's ranking of inconclusive above failing forces the team to treat inconclusive as a serious signal, not as a moment of ambiguity to be ignored.

The practical consequence is that an inconclusive exercise triggers an investigation even when all other exercises were passing. The team must determine why the inconclusive exercise was inconclusive, fix the underlying cause, and re-run the exercise before the release can claim a passing overall outcome. This discipline has caught a small number of platform-level bugs that would otherwise have been masked behind passing exercises in adjacent test runs.

The forward-compatibility commitment

How the coordination layer's outputs grow over time without breaking the consumers that already depend on them.

05

PART FIVE · THE PROBLEM OF SCHEMA EVOLUTION

Why output schemas have to grow.

The coordination layer produces structured outputs in two places: the protocol messages flowing between coordinator and participants, and the machine-readable data feeds published on the platform's website. Both of these output surfaces have downstream consumers, and both of them need to grow over time as the platform's capabilities expand. The challenge is to grow them without breaking the consumers that already depend on the current shape.

The challenge is real because the consumers are not all under our control. The protocol messages are consumed by independent participants, written by independent teams, on schedules independent of ours. The data feeds are consumed by external automation we do not know about. Any change to either output surface that breaks consumers would be visible immediately to those consumers and would, justifiably, erode their trust in our commitment to stability.

Our approach to this problem is a discipline we call forward compatibility. The discipline is simple to state: any change to an output surface must be an addition rather than a modification, and any addition must be tolerable to consumers that do not know about it. New fields can be added; old fields cannot be removed, renamed, or changed in type. New message variants can be added; old variants must continue to be supported. The discipline is asymmetric: we promise that consumers will not be broken by our changes, but we do not require consumers to take advantage of new capabilities.

Forward compatibility is enforced both by review and by automation. The review portion consists of an explicit check, on every release, that no output-surface change violates the discipline. The automation portion consists of a set of explicit tests that exercise the consumer side of every output surface, including the case in which the consumer is reading a newer output than it knows about; the tests verify that the consumer tolerates the unknown fields without error. We have a small number of these tests for each output surface, and they have caught one real regression during the platform's lifetime.

PART FIVE · AN EXAMPLE OF THE DISCIPLINE IN ACTION

A concrete example of forward-compatible growth.

To make the discipline concrete, we describe in this section the specific way one of our output surfaces will grow during the next phase. The example concerns the data feed that reports the platform's most recent measurement results, and the specific change is the addition of a new field describing how much of each tolerance budget was consumed by the most recent run.

Before the change, each entry in the data feed contains the exercise identifier, the outcome, the measurements, and the tolerance budgets that applied. A consumer reading the feed sees these fields and processes them according to its own logic. After the change, each entry additionally contains a new field describing the remaining headroom on each tolerance budget, expressed as a fraction between zero and one.

The discipline applies to this change in two ways. First, the new field is an addition: nothing else in the entry changes. A consumer that knows nothing about the new field will see an extra field it does not recognise and, if it was written under forward-compatibility expectations, will ignore the field and proceed as before. Second, the existing version marker for the data feed remains unchanged. The discipline is explicit that adding fields does not require bumping the version marker; bumping the marker is reserved for incompatible changes, which forward-compatible additions are not.

The automation that enforces the discipline runs before the change is allowed to ship. The test exercises a consumer reading both an old entry and a new entry, and verifies that the consumer produces the same output for the parts of the entry it knows about, regardless of whether the new field is present. The test also verifies that the consumer does not error or emit a warning when the new field is present. If either of these checks fails, the change is rejected and the release does not proceed.

This example may seem mundane, but it represents the daily discipline by which the platform grows. Every release that touches an output surface passes through this same review and automation. Over the months and years to come, the cumulative effect of the discipline will be that consumers of the platform's outputs can rely on them across long time horizons without being forced to upgrade their own code on our schedule.

The integrity guarantees

How the coordination layer's outputs become signed, logged, and independently verifiable.

06

PART SIX · FROM MEASUREMENT TO EVIDENCE

The path from a measurement to a signed artefact.

We have described, in the previous parts, how the coordination layer drives participants through test exercises and collects measurements from them. We have not yet described how those measurements become signed artefacts that auditors can verify. This part of the brief describes the path from raw measurement to signed artefact, in enough detail that a reader who wants to understand the integrity chain can follow it from end to end.

The path has five steps. The first step is the collection of measurements during an exercise, as described in the earlier parts. The coordinator stores the measurements in a structured form that captures enough context that the measurements could later be replayed or audited.

The second step is the aggregation of the measurements into a structured summary covering a particular time window, typically a calendar quarter. The summary contains the per-exercise outcomes, the per-exercise measurements, the overall aggregated outcome, and a small amount of metadata identifying the platform's version, the recorded change that produced the summary, and the time at which the summary was produced.

The third step is the production of a human-readable narrative document that describes the summary in plain language. The narrative is intended for a non-technical reader; it includes the summary's most important findings, the context the findings should be interpreted in, and pointers to the raw machine-readable summary for any reader who wants the underlying data. Crucially, the narrative document includes, embedded in its structured metadata, a cryptographic fingerprint of the machine-readable summary, which is what allows a verifier to confirm that the two documents describe the same underlying data.

The fourth step is the signing of the narrative document under the ephemeral-signing scheme we described at length in the main brief. The platform's automated build pipeline produces a short-lived cryptographic key pair, uses it to sign the narrative document, submits the signature to the public transparency ledger, and discards the private half of the key pair. The result is a small witness file that proves the narrative document was produced by our build pipeline at a known moment.

The fifth and final step is the publication of the three files, the narrative document, the machine-readable summary, and the witness file, to the platform's public website. The publication is performed by the same build pipeline that produced the documents, and it is subject to all of the change-control rules we described in the main brief. From this moment on, the three files are publicly available, immutable, and independently verifiable by anyone who has the auditor's verification command.

PART SIX · THE AUDITOR'S VERIFICATION, AGAIN

What the auditor does, step by step.

The auditor's verification process for a coordination-layer artefact is the same five-check sequence we described in the main brief, and we do not repeat the full description here. We do, however, want to give a coordination-layer-specific perspective on what each check confirms, because the coordination layer's specific properties affect what an auditor learns from a successful verification.

The first check confirms that the narrative document and the machine-readable summary describe the same underlying data. For coordination-layer artefacts, this check confirms that the human-readable claims in the narrative are consistent with the numerical results in the summary; it is not possible to slip a discrepancy between the two without the check failing.

The second check confirms that the signature on the witness file is mathematically valid. For coordination-layer artefacts, this confirms that the artefact was signed by some party using the ephemeral-signing scheme; it does not yet confirm that the party was us, only that the signature is well-formed.

The third check confirms that the signer's identity matches the frozen identity we have publicly committed to. For coordination-layer artefacts, this is the check that confirms the signer was actually us, by matching the ephemeral certificate's identity claim against the regular expression we have published. A successful match means an outside party can be certain that the artefact came from our build pipeline, not from any other source.

The fourth check confirms that the signature appears in the public transparency ledger at a specific position, verified offline against the inclusion proof embedded in the witness. For coordination-layer artefacts, this is what gives the auditor confidence that the signature was produced at the time it claims to have been produced; the ledger's timestamp cannot be rewritten without rewriting every subsequent entry.

The fifth check confirms that the artefact carries one of the two markers identifying which of our two reporting chains produced it. For coordination-layer artefacts, the marker will be the benchmark-chain marker, because the coordination layer's outputs are benchmarks rather than service-level commitments. An auditor who is verifying coordination-layer artefacts can use the marker to confirm they are looking at the right kind of evidence.

Closing

Summary, glossary, contact.

07

PART SEVEN · SUMMARY

The coordination layer, in four sentences.

Having taken you through six parts and many thousands of words, we owe you a compressed restatement, in case you would find it useful for your own follow-up reading. The four sentences below summarise the coordination layer's most important properties.

First: the coordination layer is a federation of independent participants, talking to a central coordinator over a structured protocol with five phases, frozen for at least the next six minor releases of the platform.

Second: the layer tolerates five specific failure modes by design, each of which has its own test exercise in the platform's catalogue and each of which is exercised by the continuous verification pipeline on every release.

Third: the layer's outputs combine through a deliberate worst-case aggregation rule into a single overall outcome in the four-valued closed vocabulary the platform commits to, and the aggregation is conservative in a specific sense we have described in detail.

Fourth: the layer's outputs are published as signed artefacts on a quarterly cadence, each artefact independently verifiable by any outside party through the same five-check sequence described in the main brief.

PART SEVEN · GLOSSARY

Glossary.

We collect here the most important coordination-layer-specific terms, in case the reader finds them useful as a reference. The glossary is intentionally short; the broader platform glossary is in the main brief.

The coordinator. The central process that drives a test exercise by sending instructions to participants, collecting their measurements, and aggregating their outcomes.

A participant. Any process that implements the coordination protocol on its server side and that engages with the coordinator during a test exercise.

A phase. One of the five segments of the coordination protocol: registration, exercise, measurement, voting, and withdrawal.

An outcome. The judgement the platform produces about a test exercise. Outcomes take one of four values: passing, warning, failing, or inconclusive.

The tolerance budget. The amount of drift from a reference value that a particular measurement is allowed to absorb before producing a warning or a failing outcome.

Adversarial behaviour. The failure mode in which a participant returns deliberately wrong answers despite being alive and reachable. The platform tolerates this through the voting phase.

The voting phase. The phase in which participants vote on what they believe the outcome should be, used to detect adversarial behaviour.

The forward-compatibility discipline. The rule that any change to an output surface must be an addition rather than a modification, and that consumers must tolerate additions they do not know about.

The aggregation rule. The mathematical rule by which per-exercise outcomes are combined into a single overall outcome. The platform uses the worst-case rule, in which the overall outcome is the worst of the per-exercise outcomes.

CONTACT

The coordination layer's engineering team is part of the broader engineering leadership team. Contact details are in the main brief. The coordination layer's source code is in the same public repository as the rest of the platform's code, and any reader who wants to inspect it can do so without our intervention.